



МОСКОВСКИЙ УНИВЕРСИТЕТ ИМЕНИ С.Ю. ВИТТЕ

Факультет Информационных технологий
Кафедра Кафедра информационных систем
Направление подготовки/Специальность Прикладная информатика

ОТЧЕТ

о прохождении

Производственная практика

/вид практики/

Эксплуатационная практика

/тип практики/

Студента Коломиеца Филиппа Андреевича
(фамилия, имя, отчество)

Место прохождения практики ЧОУВО «МУ им. С.Ю. Витте»

Период прохождения практики с 20.04.2026 г. по 31.05.2026 г.

г. Москва 2026 г.

Оглавление

Оглавление	2
Введение	3
1. Основная часть	4
2. Анализ предметной области	4
2.1. Выбор и настройка среды разработки Python	4
2.2. Анализ применения нейросетей для обработки научных текстов	4
2.3. Анализ методов поиска утверждений о существовании и единственности	5
2.4. Анализ структуры научных pdf документов	5
2.5. Вывод по разделу	6
3. Разработка системы подготовки датасета	7
3.1. Разработка структуры программы	7
3.2. Извлечение и обработка текста из pdf документов	8
3.2.1. Получение и чтение pdf файлов	8
3.2.2. Очистка и нормализация текста	9
3.2.3. Формирование текстовых фрагментов	10
3.3. Реализация поиска утверждений о существовании и единственности	11
3.4. Использование нейросети для анализа текста	12
3.4.1. Подключение модели через Ollama	13
3.4.2. Определение названия статьи и авторов	14
3.5. Формирование итогового датасета	15
3.6. Анализ результатов работы программы	16
3.7. Вывод по разделу	18
4. Заключение	19
5. Список использованной литературы	20

Введение

Производственная практика проходила в Московском университете им. С.Ю. Витте на кафедре информационных систем и технологий. В ходе практики рассматривались возможности применения нейросетевых технологий для обработки научных текстов и подготовки специального датасета.

Основной задачей работы являлось создание программы на языке Python для извлечения текстовой информации из научных pdf документов. Уделялось внимание поиску утверждений о существовании и единственности решений в математических статьях, а также формированию структурированного датасета для дальнейшего анализа.

Целью практики являлось получение практических навыков разработки программных решений с применением методов интеллектуального анализа данных и инструментов обработки текста.

Для достижения поставленной цели были определены следующие задачи:

- закрепление теоретических знаний;
- изучение возможностей Python для обработки научных документов;
- разработка программы для извлечения и очистки текста;
- реализация поиска тематических текстовых фрагментов;
- формирование итогового датасета;
- использование нейросетевой модели для анализа содержимого статей;
- анализ результатов работы программы.

В процессе выполнения работы использовались библиотеки для обработки pdf документов, инструменты нормализации текста, а также локальная языковая модель, подключённая через Ollama.

1. Основная часть

2. Анализ предметной области

2.1. Выбор и настройка среды разработки Python

Для разработки программы был выбран язык Python, так как он хорошо подходит для обработки текста и работы с данными. Кроме этого, для Python существует большое количество библиотек для анализа документов и работы с нейросетями.

Разработка выполнялась в среде Pycharm. В ходе работы были установлены библиотеки fitz, natasha и ollama. Библиотека fitz использовалась для чтения pdf документов, natasha - для обработки текста, а ollama - для подключения локальной языковой модели.

Также была настроена рабочая папка проекта, содержащая pdf файлы, основной файл программы и итоговый датасет в формате JSONL. После настройки среды была проведена проверка корректности работы библиотек и обработки документов.

2.2. Анализ применения нейросетей для обработки научных текстов

Сейчас нейросети активно применяются для обработки текстовой информации. Особенно это актуально при работе с научными статьями, так как такие документы содержат большое количество сложных терминов, формул и специализированных выражений.

При помощи нейросетевых моделей можно автоматически находить нужные фрагменты текста, определять тему документа, извлекать ключевую информацию и выполнять анализ содержания статьи. Это позволяет сократить время обработки большого количества научных публикаций.

В данной работе нейросетевая модель использовалась для определения названия статьи и авторов по содержанию первой страницы документа. Подключение модели выполнялось локально через Ollama. Такой подход позволил автоматизировать часть обработки научных текстов и упростить формирование итогового датасета.

2.3. Анализ методов поиска утверждений о существовании и единственности

При анализе научных математических текстов одной из основных задач является поиск утверждений, связанных с существованием и единственностью решений. Чаще всего подобные фрагменты встречаются в формулировках теорем, лемм, доказательств и выводов.

Для поиска таких утверждений можно использовать разные методы. Одним из вариантов является полный анализ текста нейросетевой моделью, однако такой способ требует больших вычислительных ресурсов и занимает больше времени при обработке большого количества документов.

В данной работе был использован более простой подход, основанный на поиске ключевых корней слов. Для этого анализировались слова, содержащие корни “существ” и “единствен”. Такой метод позволил находить фрагменты текста, связанные с существованием решения, условиями существования и единственностью.

После нахождения нужного блока дополнительно формировался смысловой фрагмент из соседних частей текста. Это позволило сохранить связность предложений и получить более понятный результат для последующего формирования датасета.

2.4. Анализ структуры научных pdf документов

Научные pdf документы имеют сложную структуру и содержат текст, формулы, специальные символы и различные элементы оформления. Из-за этого при извлечении текста могут появляться лишние пробелы, переносы строк и ошибки в символах.

Большинство рассмотренных статей содержали название, авторов, аннотацию, основной текст и математические выражения. Основная сложность обработки заключалась в сохранении связности текста после извлечения из PDF.

Для чтения документов использовалась библиотека `fitz`, позволяющая получать текст по страницам и отдельным блокам. После извлечения выполнялась очистка и нормализация текста для дальнейшего поиска нужных фрагментов.

2.5. Вывод по разделу

В данном разделе были рассмотрены способы обработки научных текстов и особенности работы с pdf документами. Также были изучены инструменты Python, необходимые для создания программы и подготовки датасета.

Проведённый анализ помог понять, каким образом можно находить утверждения о существовании и единственности и подготавливать текст для дальнейшей обработки.

Таблица 1

Выводы по разделу 1

Выводы	Сформированные компетенции
Изучены особенности обработки научных pdf документов и подготовки текстовых данных для дальнейшего анализа	ПК-1. Способность разработки прикладного программного обеспечения, автоматизации работы с базами данных и документами, программирования бизнес-логики приложений, интеграции разнородных данных
Проведен анализ методов поиска математических оценок, границ и неравенств в научных текстах	
Выполнено исследование способов применения нейросетевых моделей при извлечении метаданных документов	ПК-7. Способностью использовать отечественные и международные стандарты при проектировании и обеспечении качества прикладного программного обеспечения

3. Разработка системы подготовки датасета

3.1. Разработка структуры программы

Разработка программы началась с создания основного класса `DatasetBuilder`. В него были вынесены все действия, необходимые для подготовки датасета, поиск pdf файлов, чтение текста, очистка данных, поиск нужных фрагментов и сохранение результата.

Такая структура была выбрана для того, чтобы программа не состояла из отдельных несвязанных частей. Все основные методы находятся внутри одного класса и работают с общими данными, папкой с файлами, списком pdf документов, ключевыми корнями слов и моделью для анализа текста.

В начале работы задаётся папка `assets`, где находятся научные статьи, а также список корней существ и единствен. После этого создаётся объект класса `DatasetBuilder`, а запуск программы выполняется через метод `run()`. Это упростило дальнейшую разработку и проверку работы программы.

```
class DatasetBuilder:  
  
builder = DatasetBuilder(folder, roots)  
  
builder.run()
```

На рисунке 1. показана общая структура разработанной программы.

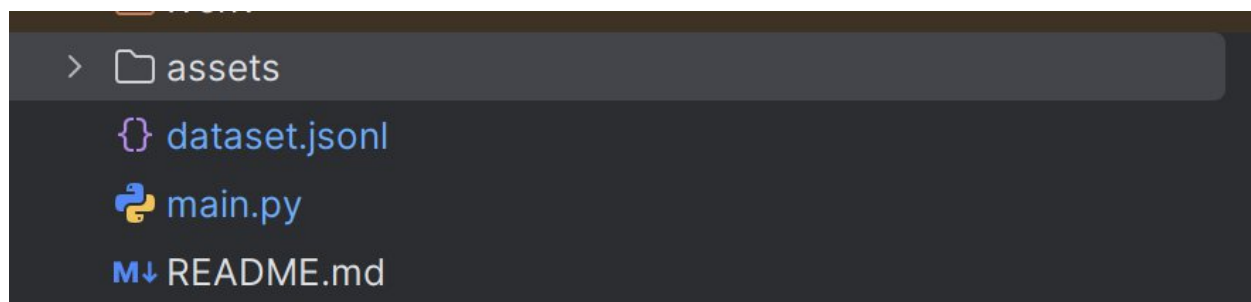


Рисунок 1.

3.2. Извлечение и обработка текста из pdf документов

3.2.1. Получение и чтение pdf файлов

На данном этапе была реализована загрузка pdf файлов из рабочей папки проекта. Для этого в программе используется папка `assets`, в которую заранее помещаются научные статьи для обработки. Программа проходит по содержимому этой папки и выбирает только те файлы, которые имеют расширение `.pdf`.

Для получения списка документов был создан метод `get_pdf_files()`. Он проверяет имена файлов, формирует полный путь к каждому pdf документу и добавляет его в общий список. После этого программа может обрабатывать все найденные статьи.

```
def get_pdf_files(self):
    files = []

    for file_name in os.listdir(self.folder):
        if file_name.endswith(".pdf"):
            full_path = self.folder + "/" + file_name

            files.append(full_path)

    return files
```

После получения списка файлов выполняется чтение каждого pdf документа. Для этого используется библиотека `fitz`, которая открывает файл и позволяет проходить по его страницам. В дальнейшем текст с каждой страницы передаётся на очистку и поиск нужных фрагментов.

```
def read_pdf(self, pdf_path):
    document = fitz.open(pdf_path)

    found_blocks = []

    first_page_text = ""
    global_index = 0
    for page_number in range(len(document)):
        page = document[page_number]
```


3.2.2. Очистка и нормализация текста

После извлечения текста из pdf документа его нельзя сразу использовать для датасета. В тексте часто остаются переносы строк, лишние пробелы, табуляции и знаки, которые мешают дальнейшему поиску нужных слов. Поэтому в программе был добавлен отдельный этап очистки.

Для очистки отдельных слов используется метод `clean_word()`. Он убирает лишние знаки препинания и приводит слово к более удобному виду для проверки.

```
def clean_word(self, dirty_word):
    symbols_to_remove = '.,!?:;()[]{}"«»-' + ""
    cleaned = dirty_word

    for symbol in symbols_to_remove:
        cleaned = cleaned.replace(symbol, "")

    return cleaned
```

Также в программе есть метод `clean_block_text()`. Он очищает уже целый текстовый блок, убирает переносы строк, табуляции и повторяющиеся пробелы.

```
def clean_block_text(self, block_text):
    cleaned_block = block_text.strip()

    cleaned_block = cleaned_block.replace("\n", " ")

    cleaned_block = cleaned_block.replace("\t", " ")

    cleaned_block = cleaned_block.replace("- ", "")

    while " " in cleaned_block:
        cleaned_block = cleaned_block.replace(" ", " ")

    return cleaned_block
```

3.2.3. Формирование текстовых фрагментов

После очистки текста программа формирует отдельные фрагменты, которые потом попадают в датасет. Это нужно для того, чтобы сохранить не просто найденное слово, а небольшой смысловой кусок текста вокруг него.

В методе `read_pdf()` каждый текстовый блок получает свой начальный и конечный индекс. За счёт этого в итоговом файле можно видеть, где примерно находился найденный фрагмент.

```
block_data = {
    "text": cleaned_block,
    "start_index": start_index,
    "end_index": end_index
}

page_blocks.append(block_data)
```

Программа проверяет каждый блок. Если в нём найден нужный корень слова, создаётся итоговый фрагмент с номером страницы, типом данных, индексами и самим текстом.

```
result_data = {
    "page": page_number + 1,
    "type": "text",
    "start_index": sense_fragment["start_index"],
    "end_index": sense_fragment["end_index"],
    "fragment": sense_fragment["text"]
}

found_blocks.append(result_data)
```

Сохраняем результат в удобном виде. В датасет попадает не весь документ, а только те части статьи, где есть нужные высказывания о существовании и единственности.

3.3. Реализация поиска утверждений о существовании и единственности

Поиск нужных утверждений в программе сделан через проверку корней слов. Для этой задачи были выбраны корни существ и единствен, так как они чаще всего встречаются в формулировках про существование решения и его единственность.

```
roots = [  
    "существ",  
    "единствен"  
]
```

Текстовый блок разбивается на отдельные слова. Слово очищается от лишних знаков, переводится в нижний регистр и сравнивается с заданными корнями.

```
def block_has_root(self, block_text):  
    words = block_text.split()  
  
    for one_word in words:  
        cleaned_word = self.clean_word(one_word)  
  
        lowered_word = cleaned_word.lower()  
  
        for root in self.roots:  
            if root in lowered_word:  
                return True  
    return False
```

Программа считает блок подходящим и добавляет его дальше в обработку. Такой способ простой, но для данной задачи он подходит, потому что в математических статьях нужные формулировки обычно содержат слова.

3.4. Использование нейросети для анализа текста

В программе нейросети используется для получения данных о научной статье. Она помогает определить название работы и авторов по тексту первой страницы pdf документа. Вот так, данные не приходится выписывать вручную для каждой статьи.

Для подключения модели используется библиотека ollama. В программе была выбрана локальная модель qwen2.5:7b.

```
self.model = "qwen2.5:7b"
```

Обращение к модели выполняется через отдельный метод ask_ollama(). В него передаётся текстовый запрос, после чего модель возвращает ответ.

```
def ask_ollama(self, prompt):
    response = ollama.generate(
        model=self.model,
        prompt=prompt
    )

    answer = response["response"]

    return answer.strip()
```

Метод используется при обработке первой страницы статьи. В запросе программе указывается, что нужно найти название статьи и авторов.

```
def find_source_info(self, first_page_text):
    prompt = (
        "Найди название научной статьи и автора или авторов. "
        "Ищи только в тексте первой страницы. "
        "Ответь строго в таком виде:\n"
        "Название: ...\n"
        "Авторы: ...\n"
        "Если данных нет, напиши unknown.\n\n"
        + first_page_text[:3000]
    )
```

3.4.1. Подключение модели через Ollama

Для работы с нейросетевой моделью в программе использовалась система Ollama. Она позволяет запускать языковые модели локально на компьютере и обращаться к ним через код.

Была подключена библиотека ollama.

```
import ollama
```

После этого в классе DatasetBuilder задаётся используемая модель. В данной работе применялась модель qwen2.5:7b.

```
self.model = "qwen2.5:7b"
```

Обращения к модели сделано отдельным методом ask_ollama(). В него передаётся текстовый запрос, после чего программа получает ответ от модели.

```
def ask_ollama(self, prompt):  
    response = ollama.generate(  
        model=self.model,  
        prompt=prompt  
    )  
  
    answer = response["response"]  
  
    return answer.strip()
```

Этот метод используется при анализе текста первой страницы статьи. Модель получает часть текста документа и пытается определить название статьи и авторов.

3.4.2. Определение названия статьи и авторов

После подключения модели была реализована автоматическая обработка первой страницы научной статьи. Это нужно для того, чтобы программа самостоятельно определяла название работы и авторов перед сохранением данных в датасет.

Решения этой задачи было создание метод `find_source_info()`. В него передаётся текст первой страницы pdf документа.

```
def find_source_info(self, first_page_text)
```

Программа формирует текстовый запрос для модели. В запросе указывается, что нужно найти название статьи и авторов только по первой странице документа.

```
prompt = (  
    "Найди название научной статьи и автора или авторов. "  
    "Ищи только в тексте первой страницы. "  
    "Ответь строго в таком виде:\n"  
    "Название: ...\n"  
    "Авторы: ...\n"  
    "Если данных нет, напиши unknown.\n\n"  
    + first_page_text[:3000]  
)
```

Запрос отправляется в модель через метод `ask_ollama()`.

```
answer = self.ask_ollama(prompt)
```

Программа разбирает полученный ответ и сохраняет найденное название статьи и авторов в отдельную структуру данных.

```
source_data = {  
    "title": title,  
    "authors": authors  
}
```

Такой способ позволил автоматически получать информацию о научной статье и сохранять её вместе с найденными текстовыми фрагментами.

3.5. Формирование итогового датасета

После поиска нужных фрагментов программа сохраняет результат в файл dataset.jsonl. Такой формат удобен тем, что каждая строка хранит данные по отдельной статье и может дальше использоваться для анализа или обучения модели.

сохранения результата делается через метод save_result(). В нём собираются имя файла, название статьи, авторы и найденные фрагменты.

```
output_data = {  
    "file": file_name,  
    "title": source_data["title"],  
    "authors": source_data["authors"],  
    "matches": result_blocks  
}
```

После этого данные переводятся в JSON-строку и записываются в итоговый файл.

```
json_line = json.dumps(output_data, ensure_ascii=False)  
output_file.write(json_line + "\n")
```

Перед началом новой обработки файл dataset.jsonl очищается, чтобы старые результаты не смешивались с новыми.

```
output_file = open("dataset.jsonl", "a", encoding="utf-8")  
output_file.close()
```

В результате формируется датасет, где для каждой статьи сохраняются основные сведения и найденные утверждения о существовании и единственности.

3.6. Анализ результатов работы программы

После запуска программы был сформирован файл dataset.jsonl, в котором сохранились результаты обработки pdf документов. В итоговый датасет попали сведения о статье, название, авторы и найденные фрагменты текста.

Пример структуры сохранённых данных выглядит следующим образом.

```
{
  "file": "assets/elibrary_11674319_64387490.pdf",
  "title": "Обобщенно-правильные системы дифференциальных уравнений",
  "authors": "Т.М. Алдибеков",
  "matches": [{
    "page": 2,
    "type": "text",
    "start_index": 3837,
    "end_index": 4061,
    "fragment": "SpA(t)dt. (5) Теорема 1. Если система (1) имеет конечные  
обобщенные показатели относительно  $q(t) \in Q$ , то для обобщенной правильности  
системы (1) относительно  $q(t)$  необходимо существование точного предела  $S(q) =$   
 $\lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t q(s) ds$ "
  }],
}
```

По результатам работы можно сделать вывод, что программа корректно находит фрагменты, связанные с утверждениями о существовании и единственности. При этом часть текста после извлечения из PDF может содержать ошибки в формулах или отдельных символах. Потому нормализация текста очень важный этап.

Полученный датасет можно использовать для дальнейшей проверки, разметки или обучения модели на задаче поиска математических утверждений. На рисунке 2. показан пример сформированного файла dataset.jsonl после выполнения программы.

main.py	dataset.jsonl	update
1	{ "file": "assets/elibrary_11674319_64387490.pdf", "title": "Обобщенно-правильные системы дифференциальных уравнений", "authors": "Т.М. 1094 ^ v	
2	{ "file": "assets/elibrary_15198846_86669137.pdf", "title": "Краевая задача общей структуры для системы ОДУ", "authors": "С. В. Исраилов, А. А. даг	
3	{ "file": "assets/elibrary_15517871_30737740.pdf", "title": "Об условиях разрешимости разностных уравнений с начальным условием из подпространства"	
4	{ "file": "assets/elibrary_15639588_77241494.pdf", "title": "unknown", "authors": "unknown", "matches": []}	
5	{ "file": "assets/elibrary_15963228_69139077.pdf", "title": "unknown", "authors": "unknown", "matches": []}	
6	{ "file": "assets/elibrary_15963235_48571298.pdf", "title": "unknown", "authors": "unknown", "matches": []}	
7	{ "file": "assets/elibrary_16460209_81388833.pdf", "title": "Обобщенное решение дифференциальных уравнений вида $y = f(x, y')$ ", "authors": "Б.П. Куф	
8	{ "file": "assets/elibrary_16525414_14596078.pdf", "title": "Ограниченные решения гиперболических уравнений с постоянными коэффициентами", "authors": "authors	
9	{ "file": "assets/elibrary_16626803_36751382.pdf", "title": "Ограниченные решения некоторых обыкновенных дифференциальных уравнений в банаховом про	
10	{ "file": "assets/elibrary_16626803_72055947.pdf", "title": "Ограниченные решения некоторых обыкновенных дифференциальных уравнений в банаховом про	
11	{ "file": "assets/elibrary_17109921_10583321.pdf", "title": "Бисекториальные операторные пучки и задача об ограниченных решениях", "authors": "А.В.	
12	{ "file": "assets/elibrary_18764233_68041818.pdf", "title": "Задача со смещением для уравнения Бицадзе-Лыкова", "authors": "Е. Ю. Арланова", "match	
13	{ "file": "assets/elibrary_19063032_55108163.pdf", "title": "О решениях системы уравнений с частными производными с двумя независимыми переменными"	
14	{ "file": "assets/elibrary_21375198_53531625.pdf", "title": "On Solvability of the Cauchy Problem for a Loaded System", "authors": "Yuriy Ya. Belov	
15	{ "file": "assets/elibrary_21875445_89569528.pdf", "title": "О одной краевой задаче Пуанкаре", "authors": "Кусковский Л. Н.", "matches": [{"page":	
16	{ "file": "assets/elibrary_22307303_82602822.pdf", "title": "unknown", "authors": "unknown", "matches": []}	
17	{ "file": "assets/elibrary_22621968_72952706.pdf", "title": "О вырождающихся неклассических системах уравнений первого порядка", "authors": "Файзие	
18	{ "file": "assets/elibrary_23143318_81906564.pdf", "title": "О НЕКЛАССИЧЕСКИХ СИСТЕМАХ УРАВНЕНИЙ С ЧАСТНЫМИ ПРОИЗВОДНЫМИ ПЕРВОГО ПОРЯДКА С ПЕРЕМЕНН	
19	{ "file": "assets/elibrary_24245826_20950295.pdf", "title": "О построении траектории одной динамической системы с начальными данными на гиперплоско	
20	{ "file": "assets/elibrary_28872241_85445535.pdf", "title": "unknown", "authors": "unknown", "matches": []}	
21	{ "file": "assets/elibrary_29009897_92756936.pdf", "title": "Аналог теоремы Боля для одного класса линейных дифференциальных уравнений в частных пр	
22	{ "file": "assets/elibrary_32357603_12262856.pdf", "title": "Краевые задачи для систем, состоящих как из алгебраических, так и дифференциальных ура	
23	{ "file": "assets/elibrary_36747251_77105538.pdf", "title": "Рассмотрен вопрос существования и единственности решения некоторых нелинейных интеграл	
24	{ "file": "assets/elibrary_37133574_71943337.pdf", "title": "Classic Solutions of Boundary Value Problems for Partial Differential Equations with Op	
25	{ "file": "assets/elibrary_42644031_56887948.pdf", "title": "Операторы и уравнения: дискретное и непрерывное", "authors": "В. Б. Васильев", "matche	
26	{ "file": "assets/elibrary_44701735_25965554.pdf", "title": "Existence and uniqueness of solutions for the system of integro-differential equations	
27	{ "file": "assets/elibrary_44902680_50340096.pdf", "title": "О единственности обобщенных решений систем дифференциальных уравнений с постоянными ко	
28	{ "file": "assets/elibrary_44902680_61393716.pdf", "title": "О единственности обобщенных решений систем дифференциальных уравнений с постоянными ко	
29	{ "file": "assets/elibrary_46570372_48815743.pdf", "title": "Теоремы единственности для уравнений в частных производных и универсальность условий р	
30	{ "file": "assets/elibrary_47395252_38889596.pdf", "title": "Разрешимость задачи Коши-Паламодова в классе обобщенных функций бесконечного порядка",	
31	{ "file": "assets/elibrary_48195078_40766885.pdf", "title": "О РАЗРЕШИМОСТИ НЕКОТОРЫХ КЛАССОВ УРАВНЕНИЙ С ПРОИЗВОДНОЙ ХИЛФЕРА В БАНАХОВЫХ ПРОСТРАНС	
32	{ "file": "assets/elibrary_48237718_37214181.pdf", "title": "О единственности продолжения обобщенного решения задачи Коши-Паламодова", "authors": "	

Рисунок 2.

3.7. Вывод по разделу

В ходе разработки была создана программа для автоматической обработки научных pdf документов и формирования датасета. Программа выполняет чтение файлов, очистку текста, поиск утверждений о существовании и единственности, а также сохранение результатов в формате JSONL.

Дополнительно была реализована работа с нейросетевой моделью через Ollama для определения названия статьи и авторов. Полученный датасет можно использовать для дальнейшего анализа научных текстов и последующей обработки данных.

Выводы по разделу 2

Таблица 2

Выводы	Сформированные компетенции
Разработана программа для автоматической обработки научных pdf документов и формирования структурированных данных Реализованы чтение pdf файлов, очистка текста и поиск математических оценок и неравенств по ключевым словам Выполнено сохранение результатов обработки в формате JSON и TXT для дальнейшего анализа данных Реализована работа с нейросетевой моделью через Ollama для определения названия статьи и авторов	ПК-1. Способность разработки прикладного программного обеспечения, автоматизации работы с базами данных и документами, программирования бизнес-логики приложений, интеграции разнородных данных ПК-8. Способность выполнять интеллектуальный анализ больших данных

4. Заключение

В ходе прохождения производственной практики была разработана программа для автоматической обработки научных pdf документов и формирования датасета. В процессе работы были изучены особенности обработки научных текстов, работа с pdf файлами и использование инструментов Python для анализа данных.

Во время практики были получены навыки работы с библиотеками для извлечения и обработки текста, а также изучены способы применения нейросетевых моделей при анализе научных публикаций. Дополнительно была реализована работа с локальной языковой моделью через Ollama для автоматического определения названия статьи и авторов.

В ходе выполнения работы были реализованы очистка и нормализация текста, поиск утверждений о существовании и единственности, а также сохранение результатов в формате JSONL. Полученный датасет может использоваться для дальнейшего анализа научных текстов и последующей обработки данных.

Таким образом, цель производственной практики была достигнута. В процессе работы были закреплены теоретические знания и получены практические навыки разработки программных решений для обработки текстовой информации и подготовки данных для дальнейшего анализа.

5. Список использованной литературы

1. Работа с pdf файлами с помощью библиотеки fitz // Хабр. – URL: [Habr PyMuPDF](#) (дата обращения: 18.05.2026).
2. Извлечение текста при помощи PyMuPDF // PythonTalk. – URL: [PythonTalk PyMuPDF](#) (дата обращения: 20.05.2026).
3. Ollama от А до Я: как выбрать модель, настроить и использовать // Хабр. – URL: [Habr Ollama Guide](#) (дата обращения: 20.05.2026).
4. Ollama documentation // Ollama. – URL: [Ollama Documentation](#) (дата обращения: 25.05.2026).
5. PyMuPDF documentation // PyMuPDF Documentation. – URL: [PyMuPDF Documentation](#) (дата обращения: 23.05.2026).
6. Обработка PDF в Python: автоматизация и парсинг документов // Skypro Media. – URL: [Skypro PDF Python](#) (дата обращения: 24.05.2026).
7. Платформа Ollama: что это и как с ней работать // Skillbox Media. – URL: [Skillbox Ollama](#) (дата обращения: 24.05.2026).